

This assignment is concerned with the implementation of a *Pick and Place* system for the myCobot 280 robot arm.

The following exercises are solved with Python and ROS 2 Jazzy. The lab combines a vision-based object detection node, custom ROS 2 messages, and a motion node for autonomous pick-and-place execution with the myCobot 280.

Before the assignment, read this document entirely and review the code structure of the provided ROS 2 packages. Make sure you understand the data flow from camera detection to robot motion execution.

Learning Objectives: Pick and Place Lab

After completing the *Pick and Place* lab, students will have an in-depth understanding of how perception and motion execution are integrated in a ROS 2-based robotic manipulation system. They will learn how to detect objects in camera images, estimate their pose, transform measurements into the robot base frame, and use the resulting information to execute autonomous pick-and-place motions.

A central learning objective is to understand the system-level architecture of the application. Students will analyze how the vision node, custom message definitions, and motion node interact via ROS 2 topics. They will learn how perception outputs are represented, published, received, filtered, and converted into concrete end-effector commands for the robot.

Students will learn how a vision node uses a trained object detector together with keypoints and a pose estimation method to determine object position. They will also understand the role of camera calibration, coordinate transformations, confidence thresholds, and workspace filtering in obtaining usable object positions for manipulation.

The lab further introduces the motion execution side of the system. Students will learn how to control the myCobot 280 in Cartesian coordinates, how to define home, approach, pick, lift, drop, and retreat motions, and how to integrate a vacuum gripper through GPIO-based control. They will also study practical issues such as hover margins, tool offsets, detection delays, color-based bin assignment, and safe return-to-home behavior.

Another learning objective is to understand how software design choices improve robustness in practical robotics. Students will examine the use of buffering detections, wait times before target selection, workspace constraints, color priorities, stack-height handling, and recalibration routines for stable execution in a real setup.

By the end of this lab, students will be able to:

- Explain the ROS 2 architecture of a vision-guided pick-and-place system.
- Use Python and ROS 2 Jazzy to integrate perception, messaging, and robot motion.

- Interpret object detections and transform them from the camera frame into the myCobot 280 base frame.
- Implement and analyze a Cartesian pick-and-place sequence with approach, pickup, transport, drop, and retreat phases.
- Use custom ROS 2 messages to exchange object pose and task-relevant information between nodes.
- Understand the role of calibration, confidence thresholds, workspace limits, and target selection logic in robust manipulation.
- Analyze how object color, stack height, and bin coordinates influence the resulting robot behavior.

These skills provide students with a practical foundation for perception-driven manipulation systems in robotics and prepare them for more advanced work in autonomous handling, visual servoing, and mobile manipulation.

Short Questions

- 1) What are the three main ROS 2 packages used in the pick-and-place system, what is the role of each one, and via which ROS 2 topic do the vision node and the motion node communicate?

- 2) The vision node defines a fixed rotation matrix $\mathbf{R}_{\text{base_cam}}$ and translation vector $\mathbf{t}_{\text{base_cam}}$ for the camera-to-base coordinate transform. Write out the full 4×4 homogeneous transformation matrix $\mathbf{T}_{\text{base_cam}}$ using these quantities.

- 3) Which fields are contained in the message `DetectedObject.msg`?

- 4) Why is a calibration step required before the vision node starts normal operation?

- 5) What is the purpose of `solvePnP` in the vision pipeline?

- 6) How does the vision node correct the raw Z-value returned by `solvePnP`, and how does it use this corrected value to determine the stack level of a detected cube?

- 7) The vacuum gripper is controlled via GPIO pins 20 and 21. What do `pump_on` and `pump_off` do, and why are two separate pins used instead of one?

- 8) What is the purpose of the workspace limits in the motion node?
- 9) Why does the motion node wait for a short time and collect several detections before selecting a target?
- 10) Explain the difference between the home position, the hover position, and the pick position.
- 11) How does the system decide to which bin a detected cube should be transported?
- 12) Name and explain three concrete robustness mechanisms that are implemented in the provided code. For each mechanism, state where in the code it appears and what failure mode it guards against.

System Overview

The provided pick-and-place application is composed of three ROS 2 packages:

- **vision**: camera-based object detection and pose estimation using a YOLO model and OpenCV,
- **mycobot_msgs**: custom ROS 2 message definitions (`DetectedObject` and `JointVelocity`),
- **mycobot_motion_v1**: motion execution, target selection, and vacuum gripper control for the myCobot 280.

The vision node captures frames from a USB camera, detects colored cubes using a trained YOLO pose model, estimates the 3D object pose with `cv2.solvePnP`, corrects the depth using a calibration factor, transforms the pose into the robot base frame, and publishes the result on the ROS 2 topic `/detected_objects`. The motion node subscribes to this topic, filters detections by workspace bounds, buffers multiple detections over a short time window, selects a target according to stack height and color priority, and executes a full pick-and-place sequence using the vacuum gripper. Figure 1 illustrates the overall data flow.

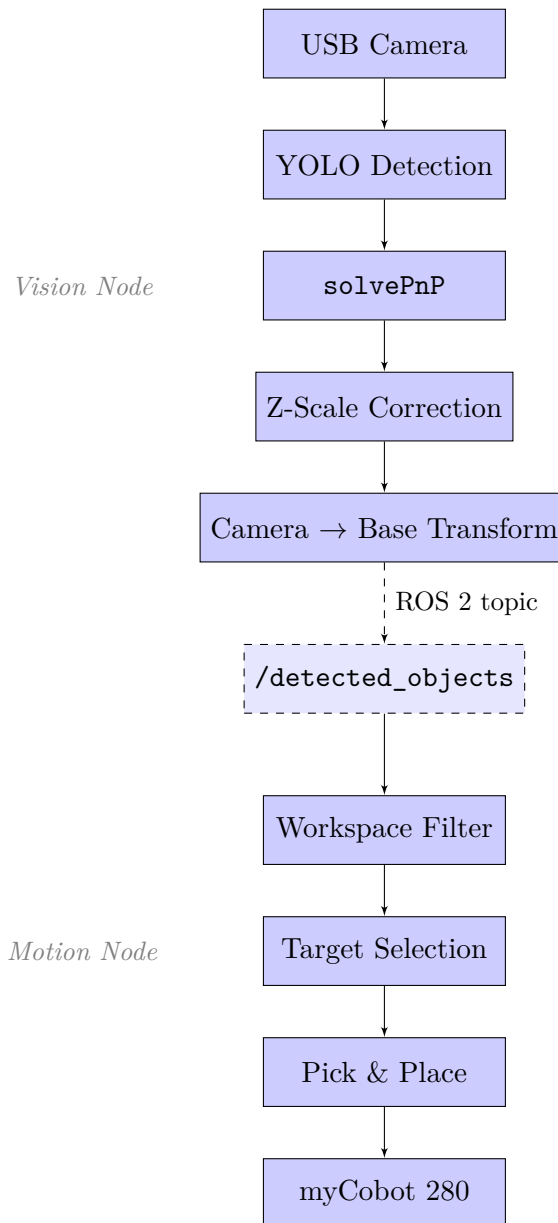


Figure 1: System architecture of the vision-guided pick-and-place application.

Vision Pipeline

The vision node uses a Python ROS 2 node to perform object detection and pose estimation from a USB camera stream. A trained YOLO pose model detects four classes of colored cubes (RED CUBE, GREEN CUBE, YELLOW CUBE, CYAN CUBE) and provides five image keypoints per detection (four corners and one center of the top face). Only detections with a confidence above the threshold $c_{\min} = 0.75$ are processed. These keypoints are used together with a known 3D object model to solve the Perspective-n-Point problem and obtain the object pose relative to the camera.

The node contains the following processing stages:

- 1) Open the USB camera and capture a frame.
- 2) Run the YOLO model on the frame with confidence threshold $c_{\min}$.
- 3) Extract bounding boxes, class labels, confidences, and keypoints.
- 4) Apply `cv2.solvePnP` to estimate the object pose in the camera frame.
- 5) Correct the estimated depth t_z using the calibration scale factor.
- 6) Estimate the cube stack level from the corrected depth.
- 7) Transform the object pose from camera coordinates into the robot base frame.
- 8) Publish the object position and color as a `DetectedObject` ROS 2 message.

Camera Intrinsics and Object Model

The vision node uses a calibrated USB camera with the following intrinsic matrix:

$$\mathbf{K} = \begin{bmatrix} 756.78 & 0 & 327.93 \\ 0 & 751.59 & 224.30 \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

and distortion coefficients $\mathbf{d} = [0.319, -2.049, -0.005, 0.007, 3.392]$.

The known 3D model of each cube face is defined by five keypoints on the top face at $Z = 0$, with a half-side length $s = 0.02$ m:

$$\mathbf{P}_i \in \left\{ \begin{bmatrix} -s \\ -s \\ 0 \end{bmatrix}, \begin{bmatrix} s \\ -s \\ 0 \end{bmatrix}, \begin{bmatrix} s \\ s \\ 0 \end{bmatrix}, \begin{bmatrix} -s \\ s \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} \quad (2)$$

These five points correspond to the four corners and the center of a 0.04×0.04 m cube face. The YOLO model is trained to detect these keypoints in image coordinates for each visible cube.

Pose Estimation with solvePnP

Given the five known 3D object points $\mathbf{P}_i$ and their corresponding 2D image projections $\mathbf{p}_i = [u_i, v_i]^T$ detected by YOLO, the Perspective-n-Point (PnP) problem determines the rotation vector $\mathbf{r}$ (Rodrigues form) and translation vector $\mathbf{t}$ of the object relative to the camera. The underlying projection equation is

$$s \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} \quad (3)$$

where s is a scale factor, $\mathbf{K}$ is the camera intrinsic matrix from Eq. (1), $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is the rotation matrix obtained from the Rodrigues vector $\mathbf{r}$ via `cv2.Rodrigues`, and $\mathbf{t} = [t_x, t_y, t_z]^T$ is the translation vector. The OpenCV function `cv2.solvePnP` solves this system and returns `rvec` and `tvec`, which describe the object pose in the camera coordinate frame.

The following code from the vision node shows the PnP call:

```
# Python -- solvePnP call in the vision node
success, rvec, tvec = cv2.solvePnP(
    object_points,      # known 3D model points (5 x 3)
    image_points,       # detected 2D keypoints (5 x 2)
    camera_matrix,      # intrinsic matrix K
    dist_coeffs         # distortion coefficients
)
R_cam_obj, _ = cv2.Rodrigues(rvec)
```

Z-Scale Calibration and Stack Level Detection

The raw depth estimate t_z from `solvePnP` is not sufficiently accurate for reliable manipulation. The vision node therefore includes a calibration procedure in which a single cube is placed in the workspace and five depth samples are captured. The average raw depth z_{ref} is used together with the known true depth $Z_{\text{one}} = 0.36$ m (the robot base frame Z coordinate for one cube on the table) to compute a scale factor:

$$f_{\text{scale}} = \frac{Z_{\text{one}}}{z_{\text{ref}}} \quad (4)$$

Each subsequent raw depth measurement is then corrected by

$$z_{\text{corrected}} = z_{\text{raw}} \cdot f_{\text{scale}} \quad (5)$$

From the corrected depth, the cube stack level ℓ is estimated. A higher stack appears closer to the camera and thus produces a smaller depth value. The level is computed as

$$\Delta z = z_{\text{corrected}} - Z_{\text{one}}, \quad \ell = \max\left(0, \text{round}\left(\frac{-\Delta z}{h_{\text{cube}}}\right)\right) \quad (6)$$

where $h_{\text{cube}} = 0.04\text{m}$ is the height of one cube. The absolute Z coordinate used for the base-frame transformation is then

$$Z_{\text{abs}} = Z_{\text{one}} - \ell \cdot h_{\text{cube}} \quad (7)$$

The corresponding code in the vision node is:

```
# Python -- Z correction and level detection
z_pnp_corrected = z_pnp_raw * self.z_scale_factor

delta_z = z_pnp_corrected - Z_ONE_CUBE
level = round(-delta_z / CUBE_HEIGHT)
level = max(0, level)

Z_absolute = Z_ONE_CUBE - level * CUBE_HEIGHT
```

Camera-to-Base Coordinate Transform

The object pose estimated by `solvePnP` is expressed in the camera coordinate frame. To command the robot, this pose must be transformed into the robot base frame. The transformation is defined by a 4×4 homogeneous matrix $\mathbf{T}_{\text{base}}^{\text{cam}}$ composed of a rotation matrix $\mathbf{R}_{\text{base}}^{\text{cam}}$ and a translation vector $\mathbf{t}_{\text{base}}^{\text{cam}}$.

$$\mathbf{T}_{\text{base}}^{\text{cam}} = \begin{bmatrix} \mathbf{R}_{\text{base}}^{\text{cam}} & \mathbf{t}_{\text{base}}^{\text{cam}} \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0.170 \\ 1 & 0 & 0 & 0.000 \\ 0 & 0 & -1 & 0.390 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

The rotation $\mathbf{R}_{\text{base}}^{\text{cam}}$ maps the camera x -axis to the robot base y -axis, the camera y -axis to the base x -axis, and the camera z -axis to the negative base z -axis. The translation $\mathbf{t}_{\text{base}}^{\text{cam}} = [0.170, 0.0, 0.39]^T$ in meters describes the position of the camera origin in the robot base frame.

The object pose in the camera frame is assembled as a 4×4 homogeneous matrix $\mathbf{T}_{\text{cam}}^{\text{obj}}$ using the rotation from `solvePnP` and the corrected translation $[t_x, t_y, Z_{\text{abs}}]^T$. The object position in the robot base frame is then obtained by

$$\mathbf{T}_{\text{base}}^{\text{obj}} = \mathbf{T}_{\text{base}}^{\text{cam}} \cdot \mathbf{T}_{\text{cam}}^{\text{obj}} \quad (9)$$

and the 3D object position in the base frame is extracted as $\mathbf{p}_{\text{base}} = \mathbf{T}_{\text{base}}^{\text{obj}}[0:3, 3]$.

The following code from the vision node implements this transformation:

```
# Python -- camera-to-base transform
R_base_cam = np.array([
    [ 0,  1,  0],
    [ 1,  0,  0],
    [ 0,  0, -1]
], dtype=np.float32)

t_base_cam = np.array([0.170, 0.0, 0.39], dtype=np.float32)

T_base_cam = np.eye(4, dtype=np.float32)
T_base_cam[:3, :3] = R_base_cam
T_base_cam[:3, 3] = t_base_cam

# Per-object transform chain
T_cam_obj = np.eye(4, dtype=np.float32)
T_cam_obj[:3, :3] = R_cam_obj
T_cam_obj[:3, 3] = [tvec[0][0], tvec[1][0], Z_absolute]

T_base_obj = T_base_cam @ T_cam_obj
object_pos_base = T_base_obj[:3, 3]
```

Motion Execution

The motion node commands the myCobot 280 in Cartesian coordinates using `send_coords`, which accepts a vector $[x, y, z, r_x, r_y, r_z]$ in millimeters and degrees. Throughout the pick-and-place sequence the end-effector orientation is fixed at $[180, 0, 0]$ (pointing straight down). The object position received from the vision node is in meters and is converted to millimeters in the motion node.

The following constants define the geometry and timing of the motion:

```
# Python -- key constants in the motion node
NOZZLE_LENGTH_MM = 68.0    # TCP offset: nozzle length
HOVER_MARGIN      = 70.0    # clearance above pick height
MOVE_SPEED        = 50     # Cartesian move speed
MOTION_SLEEP      = 1.5    # delay between moves (seconds)
DROP_Z_MM         = 60.0   # Z height for drop position

HOME_COORDS = [51.3, -63.3, 412.67, -91.75, -0.63, -89.98]
INTER_COORDS = [156.4, -17.2, 247.8, 179.27, 0.21, -1.4]
```

The pick and drop heights for the end effector are computed as

$$z_{\text{pick}} = z_{\text{obj}} + l_{\text{nozzle}}, \quad z_{\text{hover}} = z_{\text{pick}} + d_{\text{hover}}, \quad z_{\text{drop}} = z_{\text{drop,base}} + l_{\text{nozzle}} \quad (10)$$

where z_{obj} is the object height in millimeters, $l_{\text{nozzle}} = 68.0 \text{ mm}$ is the nozzle length (TCP offset), $d_{\text{hover}} = 70.0 \text{ mm}$ is the hover margin, and $z_{\text{drop,base}} = 60.0 \text{ mm}$ is the fixed drop height above the table. Figure 2 illustrates the complete motion sequence.

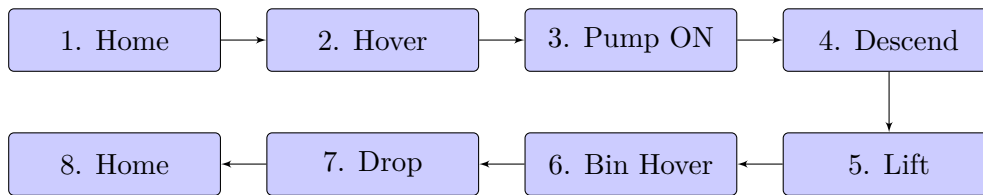


Figure 2: Pick-and-place motion sequence. The robot moves from home to hover above the object, activates the vacuum, descends to pick, lifts, moves to the color-specific bin, drops the object, and returns home.

The core motion logic from the motion node is shown below:

```

# Python -- pick-and-place sequence (abbreviated)
obj_x = self.target.x * 1000.0 # convert m to mm
obj_y = self.target.y * 1000.0
obj_z = self.target.z * 1000.0

pick_ee_z = obj_z + NOZZLE_LENGTH_MM
hover_ee_z = pick_ee_z + HOVER_MARGIN

# 1. Approach: move above object
self.mc.send_coords([obj_x, obj_y, hover_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)

# 2. Vacuum ON
self.pump_on()

# 3. Descend to pick height
self.mc.send_coords([obj_x, obj_y, pick_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)

# 4. Lift object
self.mc.send_coords([obj_x, obj_y, hover_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)

# 5. Move to color-specific bin
bx, by = BIN_COORDS[color]
self.mc.send_coords([bx, by, hover_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)

# 6. Descend to drop height

```

```

drop_ee_z = DROP_Z_MM + NOZZLE_LENGTH_MM
self.mc.send_coords([bx, by, drop_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)

# 7. Release object
self.pump_off()

# 8. Retreat and return home
self.mc.send_coords([bx, by, hover_ee_z, 180, 0, 0], MOVE_SPEED)
time.sleep(MOTION_SLEEP)
self.go_home()

```

The return-to-home routine first moves through an intermediate position to avoid collisions, then proceeds to the home position:

```

# Python -- return to home via intermediate position
def go_home(self):
    self.mc.send_coords(INTER_COORDS, HOME_SPEED, 0)
    time.sleep(HOME_DELAY)
    self.mc.send_coords(HOME_COORDS, HOME_SPEED, 0)
    time.sleep(HOME_DELAY)

```

Vacuum Gripper Control

The end effector uses a vacuum gripper controlled through GPIO pins 20 and 21 on the Raspberry Pi. Pin 20 controls the vacuum pump (suction) and pin 21 controls the release valve. Setting both pins to 0 activates suction. Setting both pins to 1 deactivates suction and releases the object.

```

# Python -- vacuum gripper GPIO control
def pump_on(self):
    GPIO.output(20, 0) # Vacuum ON (active-low)
    GPIO.output(21, 0) # Release valve closed
    self.vacuum_on = True

def pump_off(self):
    GPIO.output(20, 1) # Vacuum OFF
    time.sleep(0.3)
    GPIO.output(21, 1) # Release valve open
    time.sleep(2.0)    # wait for object to detach
    self.vacuum_on = False

```

The vacuum is activated before the descent to the pick position and deactivated at the drop location. The 2.0s delay in `pump_off` ensures that the object has fully detached before the robot retreats.

Custom ROS 2 Messages

Two custom message types are provided in the package `mycobot_msgs`:

- DetectedObject.msg
- JointVelocity.msg

The DetectedObject message contains the estimated object position in the robot base frame (in meters) and the detected color. It is the main interface between the vision and motion nodes.

```
# DetectedObject.msg
float32 x
float32 y
float32 z
string color
```

The JointVelocity message contains six joint velocity fields for joint-level control:

```
# JointVelocity.msg
float64 j1
float64 j2
float64 j3
float64 j4
float64 j5
float64 j6
```

Workspace and Target Selection

The motion node only accepts detections inside a predefined rectangular workspace in the robot base frame (in millimeters):

$$75.0 \leq x \leq 225.0, \quad -75.0 \leq y \leq 75.0 \quad (11)$$

Detections outside these bounds are rejected to prevent the robot from attempting unreachable or unsafe motions.

When the first valid detection arrives, the motion node starts a wait timer of $T_{\text{wait}} = 2.0\text{ s}$ and collects all incoming detections into a buffer. After the wait period, one target is selected from the buffer according to a priority rule. The valid detections are sorted by descending object height (taller stacks first) and then by color priority:

```
# Python -- target selection with priority
COLOR_PRIORITY = {
    "red": 0, "yellow": 1, "green": 2, "cyan": 3
}

def choose_highest_priority_target(self, detections):
    valid = [
        d for d in detections
```

```

        if d.color in COLOR_PRIORITY and self.is_inside_workspace(d)
    ]
    if not valid:
        return None
    valid.sort(key=lambda d: (-d.z, COLOR_PRIORITY[d.color]))
    return valid[0]

```

Each detected color is assigned to a specific bin location (in millimeters):

```

# Python -- bin coordinates per color
BIN_COORDS = {
    "red":    [132.2, -147.9],
    "yellow": [245.8, -150.1],
    "green":  [115.8,  177.3],
    "blue":   [ -6.9,  173.2],
    "cyan":   [ -6.9,  173.2],
}

```

Python and ROS 2 Setup

The supplied project includes a separate requirements file for the vision package. A Python virtual environment is created outside the ROS 2 workspace, activated, and then used to install the required packages (including `ultralytics`, `torch`, `opencv-python`, `scipy`, and `numpy`). Afterwards the ROS 2 Jazzy environment and the colcon workspace are sourced and the nodes can be started.

A typical setup sequence for the vision node is:

```

python3 -m venv yoloenv
source /path/to/yoloenv/bin/activate
python -m pip install --no-cache-dir -r /path/to/requirements.txt
source /opt/ros/jazzy/setup.bash
cd ~/your_workspace
colcon build
source install/setup.bash
ros2 run vision vision_node

```

Example ROS 2 Workflow

The custom message package must be built together with the motion and vision packages. A typical ROS 2 workflow for the complete system requires two terminals:

```

# Terminal 1 -- vision node (inside virtual environment)
source /path/to/yoloenv/bin/activate
source /opt/ros/jazzy/setup.bash
cd ~/your_workspace
colcon build
source install/setup.bash
ros2 run vision vision_node

```

```
# Terminal 2 -- motion node
source /opt/ros/jazzy/setup.bash
cd ~/your_workspace
source install/setup.bash
ros2 run mycobot_motion_v1 motion_node
```

In the following you are going to analyze, run, and extend a vision-guided pick-and-place system for the myCobot 280 robot arm.

The system detects colored cubes with a camera, estimates their 3D position, converts the object pose into the robot base frame, and publishes the result on a ROS 2 topic. The motion node subscribes to this topic and executes a pick-and-place routine using a vacuum gripper. Objects are sorted into bins according to their color.

The implementation combines the following technical elements:

- YOLO-based keypoint detection,
- camera pose estimation using `solvePnP`,
- depth correction through Z-scale calibration,
- camera-to-base frame transformation via a 4×4 homogeneous matrix,
- ROS 2 topic-based communication with custom messages,
- Cartesian motion commands for the myCobot 280,
- GPIO-based vacuum gripper control on a Raspberry Pi.

Competences

After successfully completing the Pick and Place lab, students will have acquired the following competences:

- They understand the structure of a ROS 2-based pick-and-place application and can explain the interaction between perception, communication, and motion execution.
- They are able to interpret the output of a vision node and use object pose information for robotic manipulation tasks.
- They can explain how camera-based detections are transformed into robot base coordinates using a 4×4 homogeneous transformation and why calibration is essential for reliable depth estimation.

- They are able to implement and analyze a Cartesian pick, lift, transport, drop, and retreat sequence for the myCobot 280, including the role of tool offsets and hover margins.
- They understand how custom ROS 2 message types support structured communication between nodes and can create and modify such messages.
- They can evaluate practical system design choices such as confidence thresholds, workspace filtering, detection buffering, and target prioritization, and can modify these parameters to observe their effect.
- They are able to configure and use a vacuum gripper via GPIO-based control in a pick-and-place workflow.
- They can reason about robustness issues in real robotic systems, including stack height estimation, calibration procedures, safety stops, and return-to-home behavior.